==========

ERP System Project Documentation

This is a comprehensive, modular Enterprise Resource Planning (ERP) system built with modern web technologies.

🏗 Architecture Design

The project follows a Monorepo-style structure, dividing the application into frontend and backend components within the src directory.

1. Technology Stack

- Frontend: Vue 3 (Composition API), Vite, Vue Router, Pinia (State Management).
- Backend: Node.js (with an MVC-like Service/Repository pattern), Express/Fastify (implied), Database configuration via db.js.
- Testing: Vitest for unit testing, Playwright for End-to-End (e2e) testing.
- Linting & Formatting: ESLint, Oxlint, Prettier.

2. Domain-Driven Modular Structure

- SYS (System Administration): Core settings, users, roles, menus, common codes(dictionaries), dynamic tables.

3. Backend Architecture (Logic Concentration)

Each module under src/backend is structured into:

- Controllers: Handle HTTP requests and responses.
- Services: Contain CORE business logic (Logic Heavy).
- Repositories: Handle database operations and queries.
- Models: Data schemas and domain models.

4. Frontend Architecture (UI & API Focused)

Each module under src/frontend is structured into:

- Views: Page-level components.
- Components: Reusable module-specific UI components.
- Stores: Pinia state management.
- API: Axios/Fetch wrappers for backend communication.

⚖️ Development Standards & Rules (강제 사항)

1. Model-Based Mock Data (데이터 폴백)

- 현재 테이블 설계가 미비하므로, Repository에서 데이터가 없을 경우 항상 모델에 정의된 하드코딩된 기본 데이터(Mock Data)를 반환하여 프론트엔드 개발을 지원합니다.

2. Strict Authentication (토큰 검증)

- 모든 API 요청은 반드시 로그인 토큰이 유효한지 검증해야 합니다. (공통 middleware 활용)
- 클라이언트에서 백엔드로 토큰 전송 시 Base64 등의 암호화를 거치며, 서버는 1초 미만의 중복/반복 로그인 시도를 봇으로 간주하고 즉시 차단(429) 및 DB에 이력을 로깅해야 합니다.

3. Modern UI/UX Design System Compliance (필수 CSS/UX 가이드 준수)

- 모든 UI 디자인은 반드시 `src/frontend/common/main.css`를 참고하여 최신 CSS 변수와 공통 가이드를 준수해야 합니다.
- **반응형(Responsive):** 모든 화면과 그리드는 뷰포트 크기에 맞춰 유연하게 줄어들거나 1열로 떨어지도록 (`grid-template-columns: repeat(auto-fit, minmax(250px, 1fr))`) 설계해야 합니다.
- **애니메이션 & 트랜지션 (Animations & Transitions):** 딱딱한 화면 전환은 지양합니다. 모달, 탭 전환, 카드 등장 등에 부드러운 스태거 효과(`fadeInUp`, `modalPop`), 투명도 전환(`fade-in`), 버튼의 호버/클릭 시 입체감(`transform: translateY`, `scale`)을 반드시 적용해야 합니다.
- **세련된 컴포넌트:** 데이터 테이블(`modern-data-table`)이나 입력창(`app-input`) 작성 시, 입력 선과 여백을 최소화하여 깔끔하고 투명한 형태를 지향하며, 포커스 시 부드러운 그림자(`box-shadow`)로 활성화 상태를 표현합니다. 모든 삭제/위험 버튼은 `.icon-btn.danger`나 `.btn-danger` 형태를 사용하여 시각적 구분을 명확히 해야 합니다.

4. Backend-Heavy Logic (로직 위치)

- 대부분의 비즈니스 로직, 데이터 가공 및 계산은 백엔드에서 처리합니다. 프론트엔드는 API 호출 및 UI 렌더링에 집중합니다.

5. Strict API Centralization (API 호출 중앙화 강제)

- **Vue 컴포넌트 내에서 `fetch()` 또는 `axios`를 직접 호출하는 것을 엄격히 금지합니다.**
- 모든 프론트엔드 API 호출은 반드시 각 모듈의 전용 API 디렉토리(예: `src/frontend/module_sys/api/`)에 있는 API 모듈(예: `userApi.js`)에 정의되어야 합니다.
- 컴포넌트는 오직 이 API 모듈의 함수만 임포트(`import`)하여 사용해야 하며, 서버 주소 등은 `.env` 파일을 통해 관리합니다.

6. Common Formatting (날짜 및 데이터)

- 날짜 타입이나 포맷팅(YYYY-MM-DD 등)은 공통 유틸리티를 통해 하나로 관리하고 사용합니다.

🚀 Setup & Commands

- Install: npm install
- Dev: npm run dev
- Build: npm run build
- Lint/Format: npm run lint / npm run format
- Test: npm run test:unit / npm run test:e2e

🔐 Environment Variables (.env)

The project requires a `.env` file located at `src/.env`. The following system variables must be configured:

- `DB_HOST`: Database host (e.g., localhost)
- `DB_USER`: Database user (e.g., root)
- `DB_PASSWORD`: Database password
- `DB_NAME`: Database name (e.g., study)
- `PORT`: Backend server port (e.g., 3000)
- `VITE_GOOGLE_CLIENT_ID`: Google OAuth2 Client ID
- `JWT_SECRET`: Secret key for JWT token generation

🎨 Common UI Components (src/frontend/common/components/)

1. AppButton: .btn, .btn-primary, .btn-secondary 기반 유니버설 버튼.
2. AppInput: .app-input 클래스 기반 표준 폼 입력 필드.
3. AppCard: .app-card 클래스 기반 컨텐츠 그룹화 컨테이너.
4. AppHeader & AppFooter: 레이아웃용 상단/하단 공통 바.
5. AppPageTitle: .app-page-title 기반의 페이지 메인 타이틀 컴포넌트.
6. AppGrid: flexbox 기반 그리드 시스템 (app-grid, app-grid-item).
7. AppTable: .app-table 클래스 기반 표준 데이터 테이블.
8. AppModal: .app-modal 기반의 팝업 및 다이얼로그 시스템.

==========